



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 42

Dr. Asad Mahmood
May 12th, 2026

Lecture Outline

- Announcements (Quiz Solutions, Prefinals, Final Exam Preparations)
- Outline of Previous Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.4 Measuring and Improving Cache Performance
- Outline of Today's Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.4 Measuring and Improving Cache Performance
 - 5.8 A Common Framework for Memory Hierarchy
 - Final Word

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.9 onwards (self-reading)

How Much Associativity

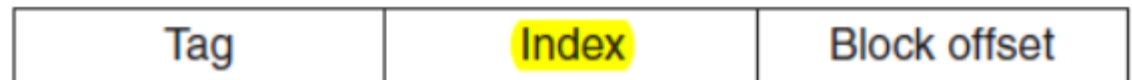
- Increased associativity decreases miss rate
 - But with diminishing returns
- Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000

Associativity	Data miss rate
1	10.3%
2	8.6%
4	8.3%
8	8.1%

Locating a block in Set Associative Cache

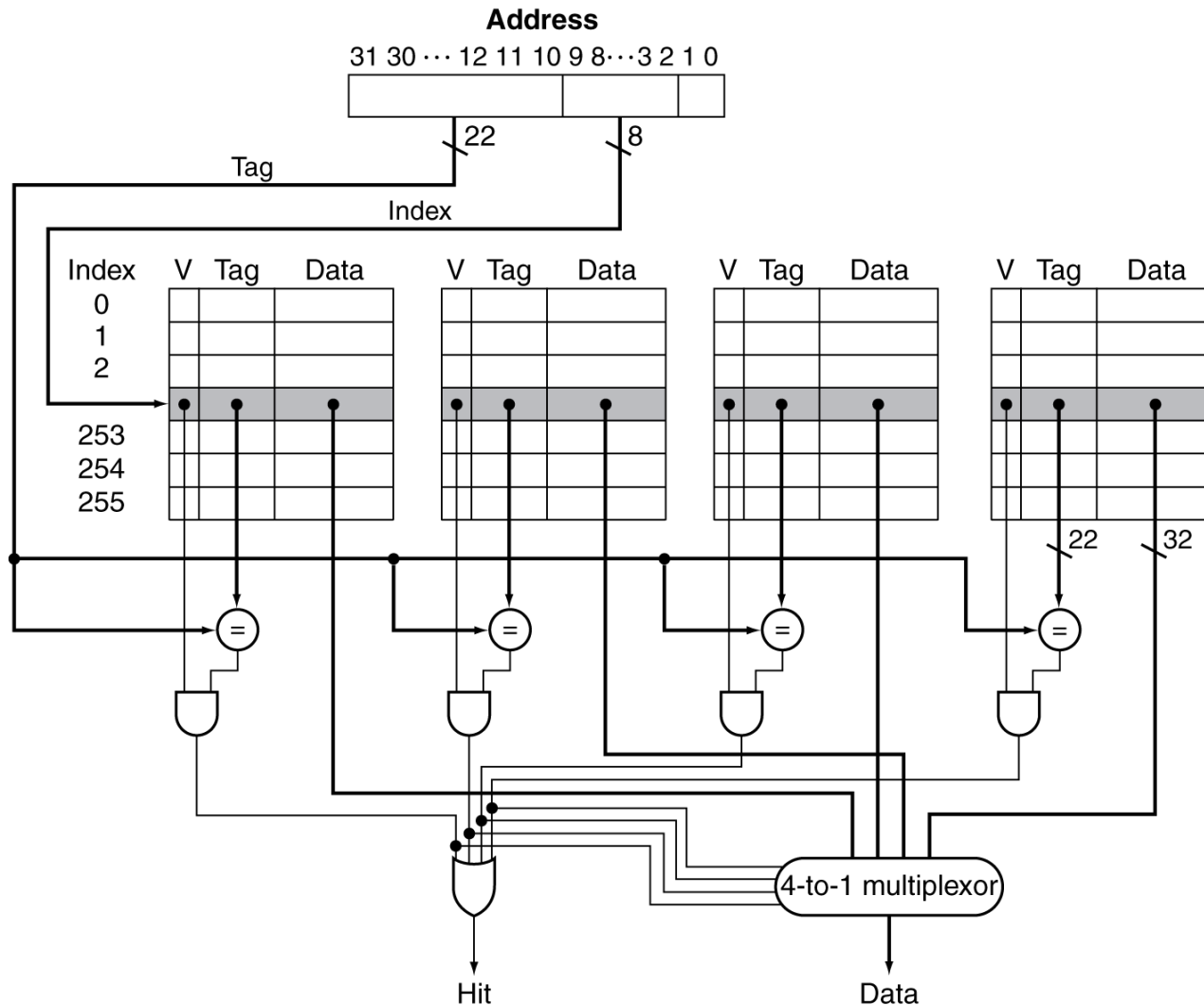
- Like direct mapped cache, each block has a tag
- The tag of every cache block within the appropriate set is checked
..... in parallel

- Three portions of the memory address



- Impact of increase in associativity on number of comparisons needed
- Impact of increase in associativity on number of Tag bits
 - E.g. a factor of 2 increase in associativity
 - Example

Set Associative Cache Organization



Replacement Policy

- Direct mapped: **no choice**
- Set associative
 - Prefer **non-valid entry**, if there is one
 - Otherwise, choose among entries in the set
- Least-recently used (LRU)
 - **Choose the one unused for the longest time**
 - Simple for 2-way, manageable for 4-way, **too hard beyond that**
- Random
 - Gives **approximately the same performance as LRU** for high associativity!

Multilevel Caches

- Primary cache attached to CPU
 - Small, but fast
- Level-2 cache services misses from primary cache
 - Larger, slower
- Main memory services L-2 cache misses
- Some high-end systems include L-3 cache

Multilevel Cache Example

- Given
 - CPU base CPI = 1, clock rate = 4GHz
 - Miss rate/instruction = 2%
 - Main memory access time = 100ns
- With just primary cache

Multilevel Cache Example

Now add L-2 cache

- Access time (for both hit and miss) = 5ns
- Reduces miss rate to main memory to 0.5%
- Penalty in case of Primary miss with L-2 hit?
- Penalty in case of Primary miss with L-2 miss?
- Effective CPI?
- Performance ratio?

Multilevel Cache Considerations

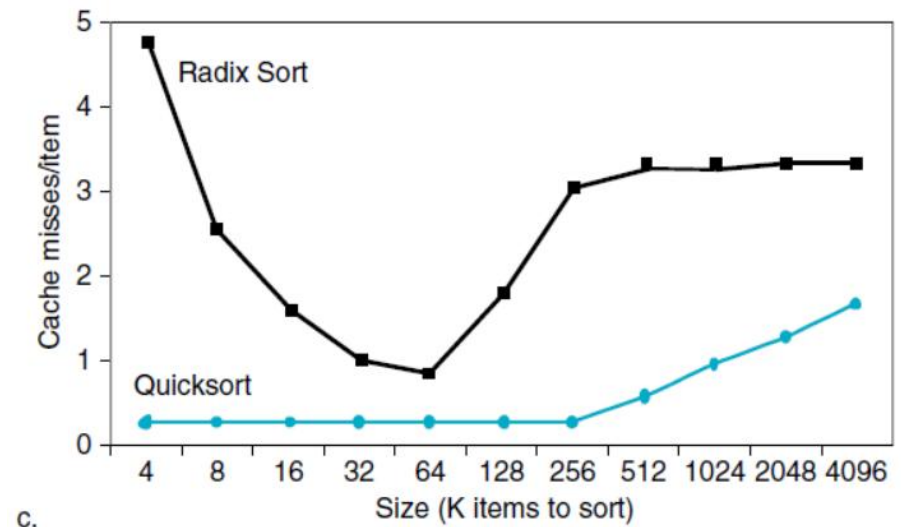
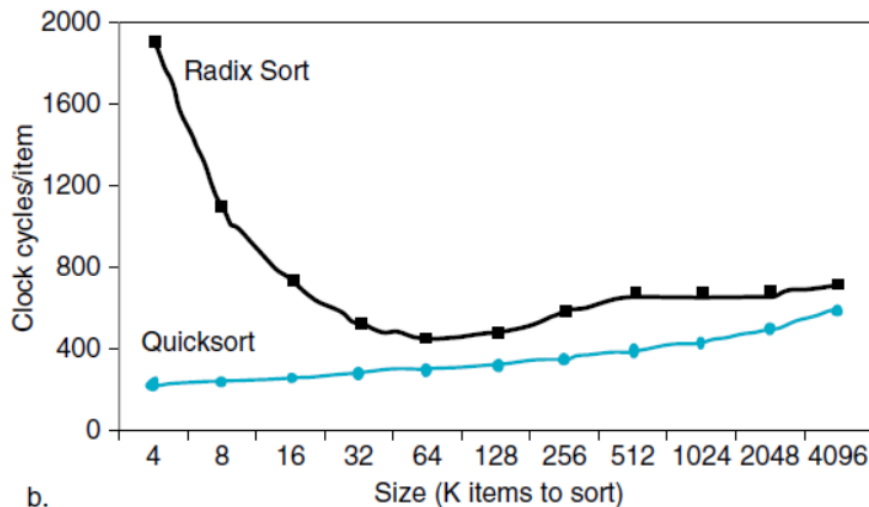
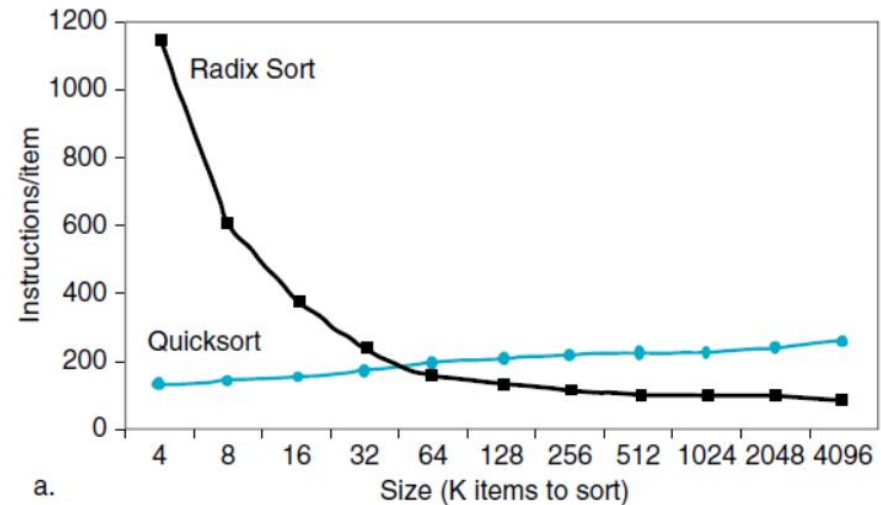
- Design considerations for a primary and secondary cache are significantly different
- Primary cache
 - Focus on minimal hit time
- L-2 cache
 - Focus on low miss rate
- Result
 - L-1 cache usually smaller than a single cache
 - L-1 block size smaller than L-2 block size

Advanced CPUs

- Out-of-order CPUs can execute instructions during cache miss
- Dependent instructions wait in reservation stations, whereas, independent instructions continue
- Effect of miss depends on program data flow
 - Much harder to analyse
 - Use system simulation

Algorithms dependence on Memory

- Cache Misses depend on memory access patterns in algorithms
 - Relevance in sorting?
 - Solution?



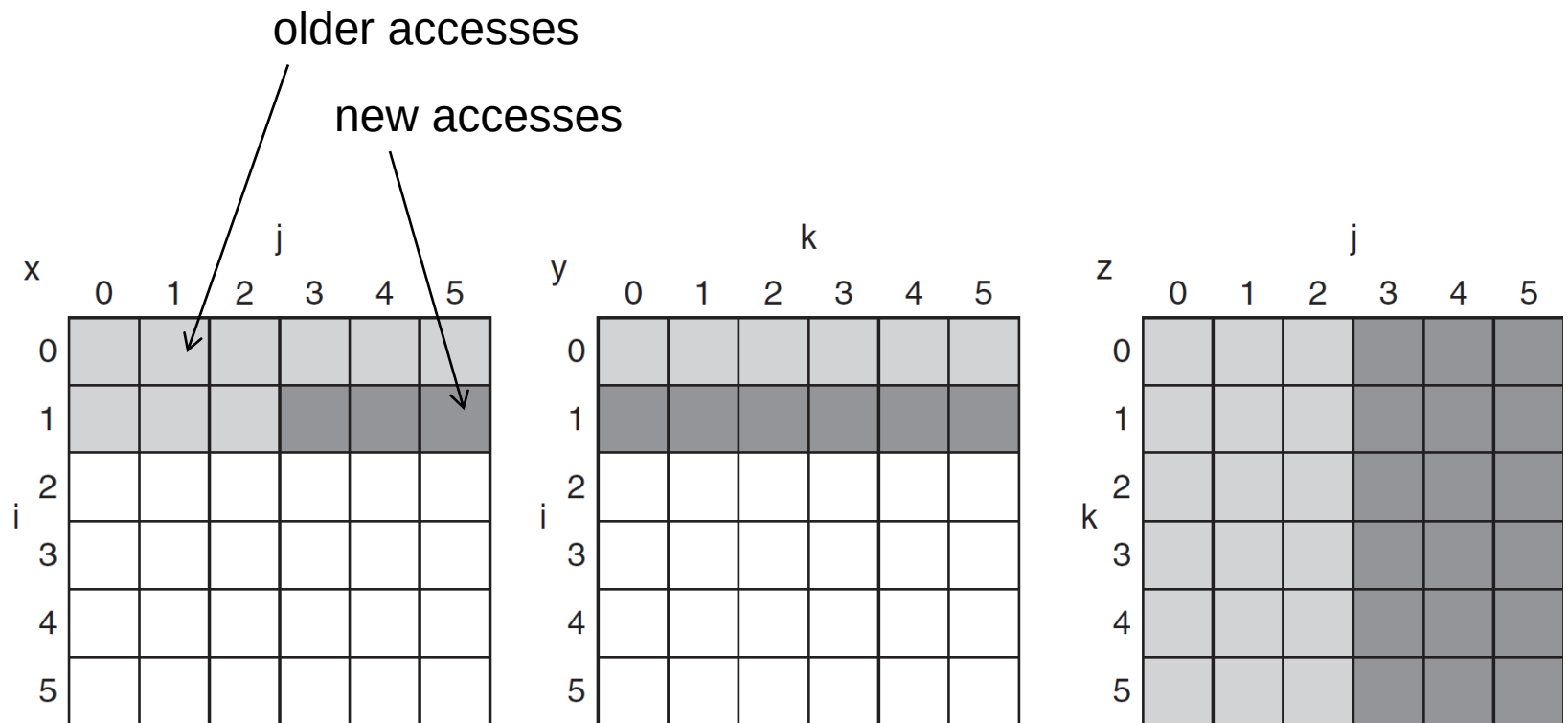
Software Optimization via Blocking

- Goal: maximize accesses to data before it is replaced
- Consider inner loops of DGEMM:

```
for (int j = 0; j < n; ++j)
{
    double cij = C[i+j*n];
    for( int k = 0; k < n; k++ )
        cij += A[i+k*n] * B[k+j*n];
    C[i+j*n] = cij;
}
```

DGEMM Access Pattern

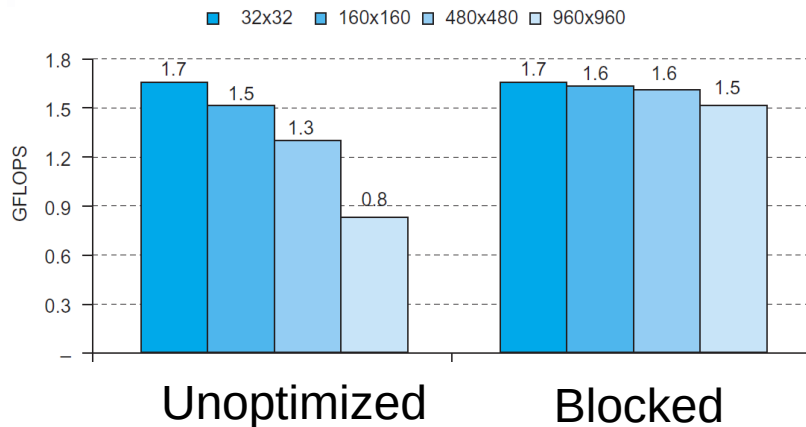
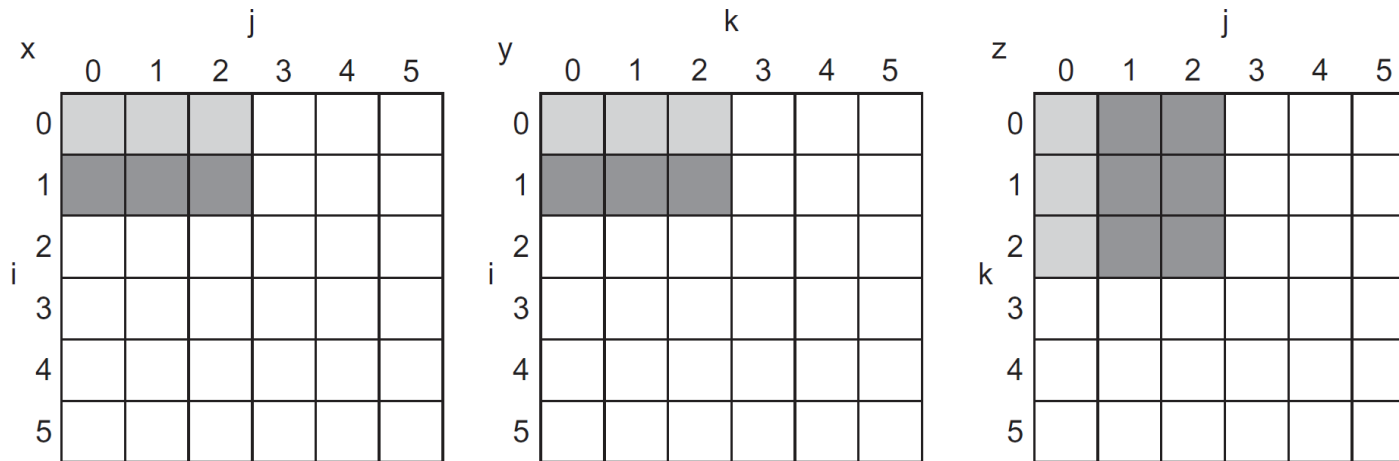
- C, A, and B arrays



Cache Blocked DGEMM

```
1 #define BLOCKSIZE 32
2 void do_block (int n, int si, int sj, int sk, double *A, double
3 *B, double *C)
4 {
5     for (int i = si; i < si+BLOCKSIZE; ++i)
6         for (int j = sj; j < sj+BLOCKSIZE; ++j)
7             {
8                 double cij = C[i+j*n]; /* cij = C[i][j] */
9                 for( int k = sk; k < sk+BLOCKSIZE; k++ )
10                     cij += A[i+k*n] * B[k+j*n]; /* cij+=A[i][k]*B[k][j] */
11                 C[i+j*n] = cij; /* C[i][j] = cij */
12             }
13 }
14 void dgemm (int n, double* A, double* B, double* C)
15 {
16     for ( int sj = 0; sj < n; sj += BLOCKSIZE )
17         for ( int si = 0; si < n; si += BLOCKSIZE )
18             for ( int sk = 0; sk < n; sk += BLOCKSIZE )
19                 do_block(n, si, sj, sk, A, B, C);
20 }
```

Blocked DGEMM Access Pattern



Conclusion: Cache-aware algorithms can outperform naive implementations even when the mathematical operations are identical!

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.9 onwards (self-reading)

The Memory Hierarchy

- Different levels of memory hierarchy, while different quantitatively, apply many *common principles*
- *Common principles* at each level lie in the answer to the following questions:
 - Q1: Where can a block be placed?
 - Q2: How is a block found?
 - Q3: What block is replaced on a miss?
 - Q4: How are writes handled?

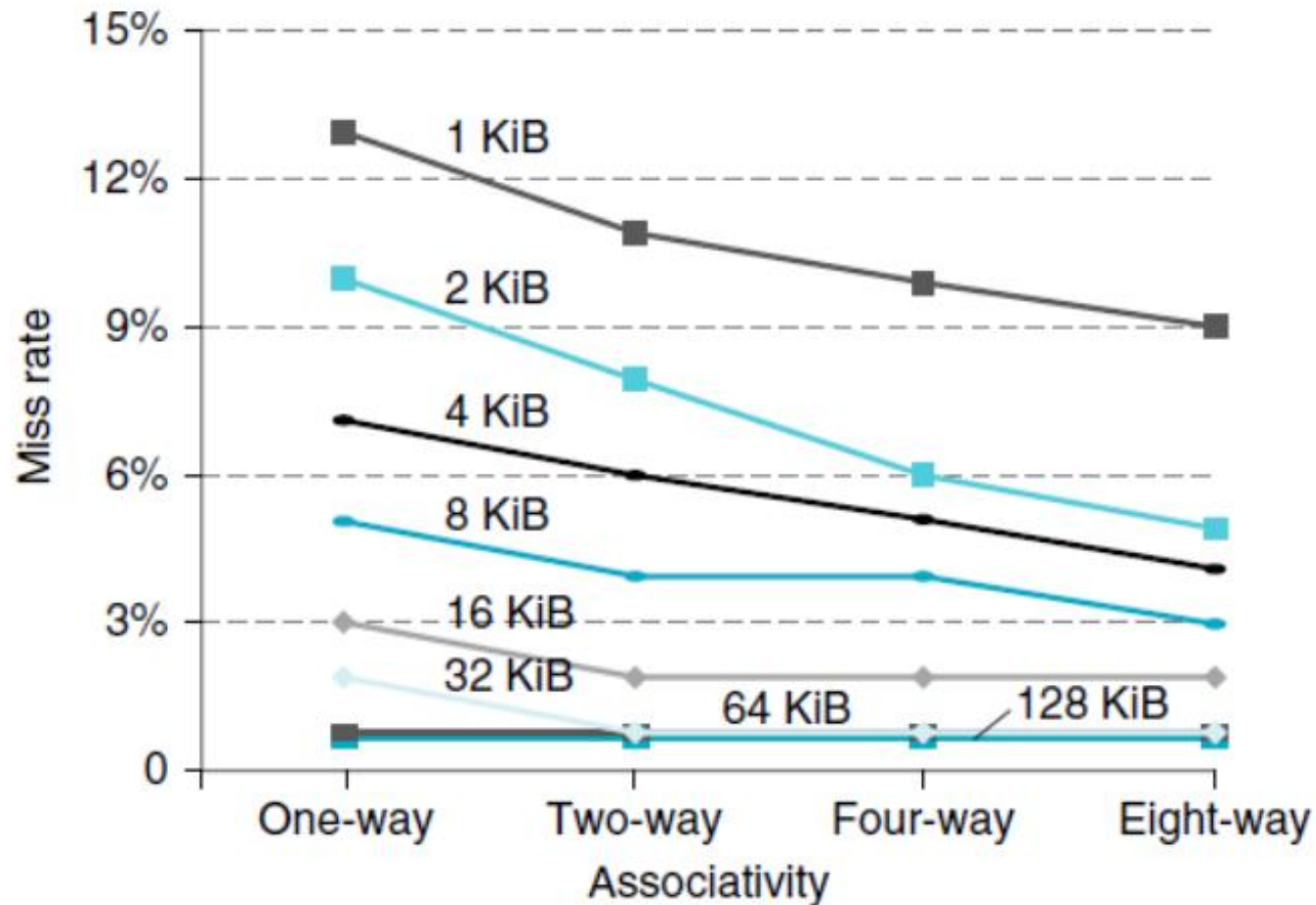
Feature	Typical values for L1 caches	Typical values for L2 caches	Typical values for paged memory	Typical values for a TLB
Total size in blocks	250–2000	2500–25,000	16,000–250,000	40–1024
Total size in kilobytes	16–64	125–2000	1,000,000–1,000,000,000	0.25–16
Block size in bytes	16–64	64–128	4000–64,000	4–32
Miss penalty in clocks	10–25	100–1000	10,000,000–100,000,000	10–1000
Miss rates (global for L2)	2%–5%	0.1%–2%	0.00001%–0.0001%	0.01%–2%

Q1: Where can a block be placed?

- Answered by the **block placement policy**
- Determined by associativity
 - Direct mapped
 - n-way set associative
 - Fully associative
- **Associativity/choices for placement**
- **Pros/Cons of higher associativity**

Scheme name	Number of sets	Blocks per set
Direct mapped	Number of blocks in cache	1
Set associative	$\frac{\text{Number of blocks in the cache}}{\text{Associativity}}$	Associativity (typically 2–16)
Fully associative	1	Number of blocks in the cache

Q1: Where can a block be placed?



Q2: How is a block found?

Associativity	Location method	Comparisons required
Direct mapped	Index	1
Set associative	Index the set, search among elements	Degree of associativity
Full	Search all cache entries	Size of the cache
	Separate lookup table	0

- **How to choose** amongst the different associativity levels?
....depends on the cost of a miss versus the cost of implementing associativity, both in time and in extra hardware

Q3: What block is replaced on a miss?

- A concern for **which type of caches?**
- Different **replacement policies** available
 1. **Least recently used (LRU)**
 - **Complex and costly** hardware for high associativity
 2. **Random**
 - Close to LRU, **easier to implement**

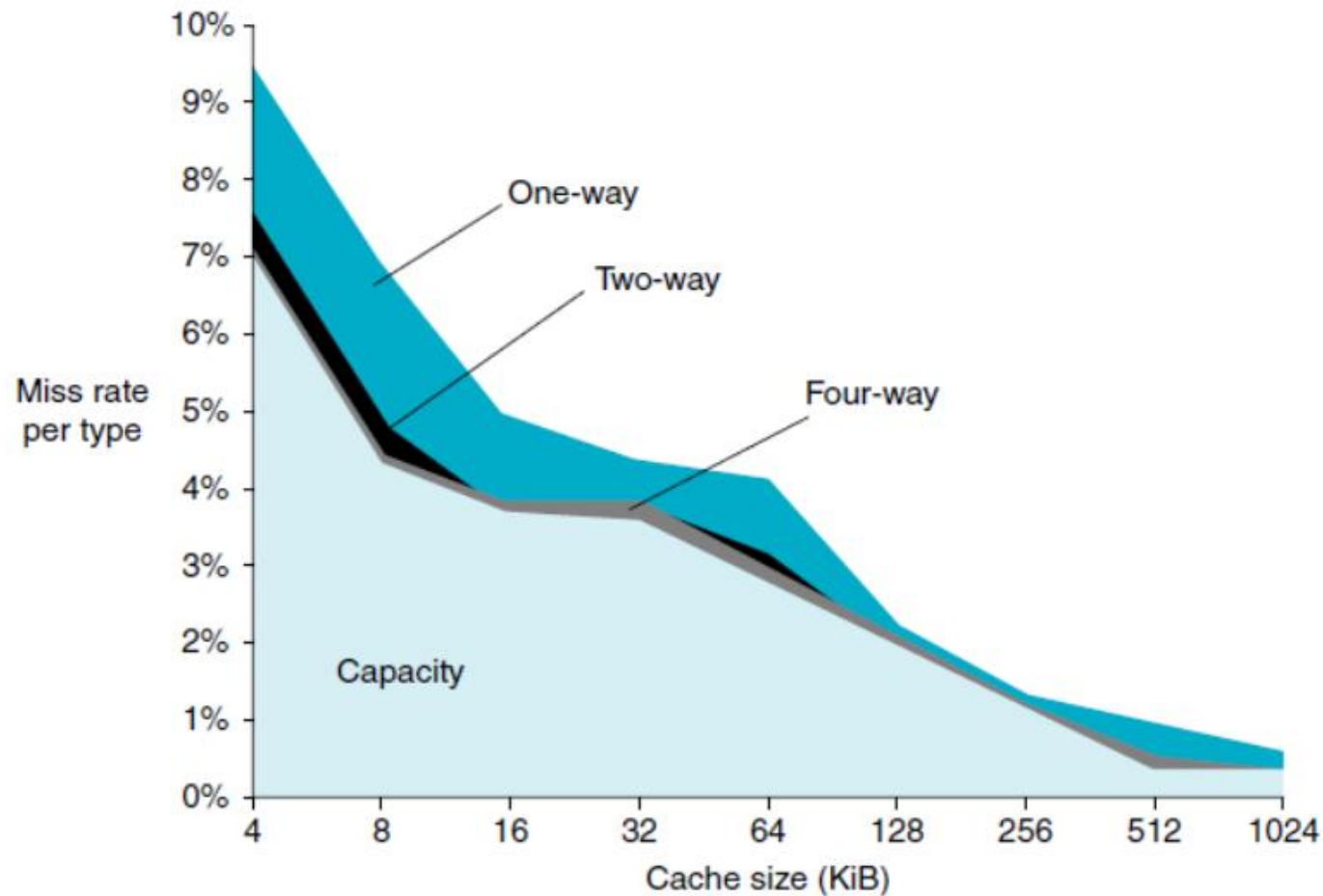
Q4: How are writes handled?

- Answered by the **write policy**
- Different **write policies** available:
 1. **Write-through**
 2. **Write-back**
- **Pros/cons**

Sources of Misses

- All misses in the cache can be classified into (*The 3 Cs*):
 1. **Compulsory misses** (aka **cold start misses**)
 - **First access** to a block
 2. **Capacity misses**
 - Due to **finite cache size**
 3. **Conflict misses** (aka **collision misses**)
 - **Due to competition** for entries in a set
 - **In what type of caches this will be present?**

Sources of Misses



Cache Design Trade-offs

Design change	Effect on miss rate	Negative performance effect
Increase cache size	Decrease capacity misses	May increase access time
Increase associativity	Decrease conflict misses	May increase access time
Increase block size	Decrease compulsory misses	Increases miss penalty For very large block size, may increase miss rate due to pollution.

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.9 onwards (self-reading)

Final Word

- Hope you enjoyed the course
- Final Exams Preparations
- Make the best of remaining life@GIKI
- Coursework and Research with me Ahead